# Tracky – Költségvetés-kezelő Rendszer Dokumentáció

## Technológiai összefoglaló

**Backend:** PHP 8.2 · Laravel 12 · Laravel Sanctum · MySQL
**Frontend:** Angular 21+ · TypeScript · CSS3
**Eszközök:** Visual Studio Code · Git

---

## Tartalomjegyzék


1. Bevezetés
2. Fejlesztői dokumentáció – Backend
   2.1. Fejlesztéshez használt eszközök és technológiák
   2.2. Adatbázis felépítés
   2.3. Mappa struktúra
   2.4. Környezeti változók (.env)
   2.5. API végpontok
   2.6. Kontrollerek és szervizek
   2.7. Autentikáció – Laravel Sanctum
   2.8. Továbbfejlesztési lehetőségek
3. Fejlesztői dokumentáció – Frontend
   3.1. Mappa struktúra
   3.2. Fejlesztéshez használt eszközök és technológiák
   3.3. Komponensek
   3.4. Szervizek
   3.5. Útvonalak és route-védők
   3.6. Adatmodellek (TypeScript interfészek)
   3.7. HTTP Interceptorok
   3.8. Továbbfejlesztési lehetőségek
4. Felhasználói kézikönyv
   4.1. Regisztráció és bejelentkezés
   4.2. Beállítások (fizetés, fix kiadások, értesítések)
   4.3. Fő képernyő – Naptár és költéskezelés
   4.4. Adminisztrátori felület
   4.5. Téma váltás
5. Telepítési útmutató
   5.1. Követelmények
   5.2. MySQL adatbázis létrehozása
   5.3. Laravel backend telepítése
   5.4. Angular frontend telepítése
   5.5. Alapértelmezett belépési adatok
   5.6. Fejlesztői parancsok

5.7. Hibaelhárítás
6. Összefoglalás

---

# 1. Bevezetés

A következő dokumentáció egy személyes költségvetés-kezelő alkalmazás, a Tracky tervezését és megvalósítását mutatja be. A fejlesztés célja egy korszerű, webalapú platform létrehozása volt, amellyel a felhasználók nyomon követhetik havi bevételeiket és kiadásaikat, rögzíthetik a napi költéseiket, beállíthatják a fix kiadásaikat és értesítéseiket, valamint adminisztrátori felületen kezelhetik a rendszer felhasználóit.

A rendszer lehetővé teszi a havi fizetés rögzítését, fix kiadások (rezsi, albérlet, stb.) kezelését, napi költések rögzítését kategóriák szerint, értesítések beállítását (pl. számlafizetési emlékeztetők), valamint egy naptár nézetben áttekinthetővé teszi a költéseket. Az alkalmazás két jogosultsági szintet különböztet meg: **Admin** (super admin) és **Felhasználó** (normál bejelentkezett user).

Technikai összefoglaló: A backend Laravel 12 / PHP 8.2 alapon REST API-t biztosít MySQL adatbázissal, Laravel Sanctum token-alapú hitelesítéssel. A frontend Angular 21+ keretrendszerben készült, standalone komponens architektúrával, CSS3 stílusozással.

---

# 2. Fejlesztői dokumentáció – Backend

## 2.1. Fejlesztéshez használt eszközök és technológiák

| Eszköz / Technológia | Verzió | Felhasználás |
|---|---|---|
| PHP | 8.2+ | Szerveroldali programozási nyelv |
| Laravel | 12.x | PHP keretrendszer – routing, ORM, middleware |
| Laravel Sanctum | 4.x | Token-alapú API hitelesítés |
| MySQL | 8.0+ | Relációs adatbázis-kezelő rendszer |
| Composer | 2.x | PHP függőségkezelő |
| DBeaver | – | Adatbázis-kezelő GUI |
| Postman | – | API végpontok tesztelése |
| Visual Studio Code | – | Fejlesztői szövegszerkesztő |
| Git | – | Verziókezelő rendszer |

## 2.2. Adatbázis felépítés

Az adatbázis MySQL 8.0 alapon fut, a táblák és kapcsolataik Laravel migrációkon keresztül kerülnek létrehozásra. Az adatbázis öt fő táblából áll.

users – Felhasználók

| Oszlop | Típus | Leírás |
|---|---|---|
| id | bigint (PK) | Egyedi azonosító |
| name | string | Felhasználónév (belépéshez) |
| email | string (unique) | E-mail cím |
| password | string (hash) | Bcrypt-tel titkosított jelszó |
| salary | integer (default: 0) | Havi fizetés |
| is_admin | boolean (default: false) | Admin jogkör |
| email_verified_at | timestamp | E-mail megerősítés időpontja |
| created_at / updated_at | timestamp | Automatikus időbélyegek |

fixed_deductions – Fix kiadások

| Oszlop | Típus | Leírás |
|---|---|---|
| id | bigint (PK) | Egyedi azonosító |
| user_id | bigint (FK) | Kapcsolat a users táblával (cascade delete) |
| name | string | Kiadás megnevezése (pl. Rezsi) |
| amount | integer | Összeg forintban |
| created_at / updated_at | timestamp | Automatikus időbélyegek |

notifications – Értesítések / Emlékeztetők

| Oszlop | Típus | Leírás |
|---|---|---|
| id | bigint (PK) | Egyedi azonosító |
| user_id | bigint (FK) | Kapcsolat a users táblával (cascade delete) |
| name | string | Értesítés megnevezése |
| amount | integer | Összeg |
| day | tinyint (1-31) | A hónap napja, amikor esedékes |
| recurring | boolean (default: true) | Ismétlődő-e minden hónapban |
| created_at / updated_at | timestamp | Automatikus időbélyegek |

expenses – Költések

| Oszlop | Típus | Leírás |
|---|---|---|
| id | bigint (PK) | Egyedi azonosító |
| user_id | bigint (FK) | Kapcsolat a users táblával (cascade delete) |
| date | date | Költés dátuma |
| amount | integer | Költés összege forintban |
| description | string (nullable) | Leírás, kategória |
| created_at / updated_at | timestamp | Automatikus időbélyegek |

## 2.3. Mappa struktúra

```
trackmate-fullstack/backend/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── AuthController.php
│   │   │   ├── UserController.php
│   │   │   ├── ExpenseController.php
│   │   │   ├── FixedDeductionController.php
│   │   │   ├── NotificationController.php
│   │   │   └── AdminController.php
│   │   └── Middleware/
│   │       └── AdminMiddleware.php
│   ├── Models/
│   │   ├── User.php
│   │   ├── Expense.php
│   │   ├── FixedDeduction.php
│   │   └── Notification.php
│   └── Providers/
│       └── AppServiceProvider.php
├── bootstrap/
│   ├── app.php
│   └── providers.php
├── config/
│   ├── app.php
│   ├── auth.php
│   ├── database.php
│   └── ...
├── database/
│   ├── migrations/
│   │   ├── 0001_01_01_000000_create_users_table.php
│   │   ├── 2026_04_23_000001_add_salary_to_users_table.php
│   │   ├── 2026_04_23_000002_add_is_admin_to_users_table.php
│   │   ├── 2026_04_23_000003_create_fixed_deductions_table.php
│   │   ├── 2026_04_23_000004_create_notifications_table.php
│   │   └── 2026_04_23_000005_create_expenses_table.php
│   └── seeders/
│       └── DatabaseSeeder.php
├── routes/
│   ├── api.php
│   └── web.php
├── public/
├── storage/
├── vendor/
├── artisan
├── composer.json
└── .env
```

## 2.4. Környezeti változók (.env)

Az alkalmazás konfigurációja a `.env` fájlból olvasódik be. A fájl sohasem kerül verziókezelésbe (`.gitignore`).

| Változó | Leírás |
|---|---|
| APP_KEY | Az alkalmazás titkosítási kulcsa (`php artisan key:generate`) |
| APP_URL | Az alkalmazás URL-je (`http://localhost:8000`) |
| DB_CONNECTION | Adatbázis driver (`mysql`) |
| DB_HOST | Adatbázis szerver host (`127.0.0.1`) |
| DB_PORT | Adatbázis port (`3306`) |
| DB_DATABASE | Adatbázis neve (`trackmate`) |
| DB_USERNAME / DB_PASSWORD | Adatbázis hitelesítési adatok |
| SANCTUM_STATEFUL_DOMAINS | Sanctum által engedélyezett domainek |

## 2.5. API végpontok

Minden védett végpont (`auth:sanctum` middleware) esetén az `Authorization` fejlécben Bearer token szükséges. A token a bejelentkezéskor kerül kiadásra.

### Publikus végpontok (hitelesítés nélkül)

| Metódus | Végpont | Leírás | Hozzáférés |
|---|---|---|---|
| POST | `/api/register` | Regisztráció új felhasználónak | Publikus |
| POST | `/api/login` | Bejelentkezés, token visszaadása | Publikus |
| GET | `/api/ping` | Szerver státusz lekérdezése | Publikus |

### Védett végpontok (Bearer token szükséges)

| Metódus | Végpont | Leírás | Hozzáférés |
|---|---|---|---|
| POST | `/api/logout` | Kijelentkezés, token törlése | Bejelentkezett |
| GET | `/api/me` | Bejelentkezett felhasználó adatai | Bejelentkezett |
| GET | `/api/user` | User profil (salary, deductions, notifications, expenses) | Bejelentkezett |
| POST | `/api/user` | User adatok mentése | Bejelentkezett |
| POST | `/api/user/salary` | Fizetés frissítése | Bejelentkezett |
| GET | `/api/expenses` | Költések listázása | Bejelentkezett |
| POST | `/api/expenses` | Új költés rögzítése | Bejelentkezett |
| PUT | `/api/expenses/{expense}` | Költés módosítása | Bejelentkezett (saját) |
| DELETE | `/api/expenses/{expense}` | Költés törlése | Bejelentkezett (saját) |
| GET | `/api/deductions` | Fix kiadások listázása | Bejelentkezett |
| POST | `/api/deductions` | Új fix kiadás | Bejelentkezett |
| PUT | `/api/deductions/{deduction}` | Fix kiadás módosítása | Bejelentkezett (saját) |
| DELETE | `/api/deductions/{deduction}` | Fix kiadás törlése | Bejelentkezett (saját) |

| GET | `/api/notifications` | Értesítések listázása | Bejelentkezett |
| POST | `/api/notifications` | Új értesítés | Bejelentkezett |
| PUT | `/api/notifications/{notification}` | Értesítés módosítása | Bejelentkezett (saját) |
| DELETE | `/api/notifications/{notification}` | Értesítés törlése | Bejelentkezett (saját) |

Admin végpontok (Bearer token + is_admin = true)

| Metódus | Végpont | Leírás | Hozzáférés |
|---|---|---|---|
| GET | `/api/admin/users` | Összes felhasználó listázása | Admin |
| GET | `/api/admin/users/{user}` | Egy felhasználó részletei | Admin |
| POST | `/api/admin/users` | Új felhasználó létrehozása | Admin |
| PUT | `/api/admin/users/{user}` | Felhasználó módosítása | Admin |
| DELETE | `/api/admin/users/{user}` | Felhasználó törlése | Admin |
| GET | `/api/admin/stats` | Statisztikák (összesített adatok) | Admin |

## 2.6. Kontrollerek és szervizek

AuthController
A regisztrációs és belépési folyamatot kezeli. A `register()` metódus validálja az adatokat, hash-eli a jelszót, majd létrehozza a felhasználót és visszaad egy Sanctum tokent. A `login()` metódus ellenőrzi a felhasználónevet és jelszót, majd szintén tokent ad vissza. A `logout()` törli az aktuális tokent. A `me()` visszaadja a bejelentkezett felhasználó teljes adatait a kapcsolódó rekordokkal (fixedDeductions, notifications, expenses) eager loading segítségével.

UserController
A felhasználói profil kezelését végzi. A `show()` metódus visszaadja az aktuális user salary, fixedDeductions, notifications és expenses adatait. A `update()` metódus szinkronizálja a kapcsolódó táblákat (törli a régieket és újra létrehozza a kapott adatok alapján). A `updateSalary()` külön végponton frissíti csak a fizetést.

ExpenseController
A költések teljes CRUD-ját biztosítja. Minden műveletnél ellenőrzi, hogy a kért expense a bejelentkezett felhasználóhoz tartozik-e, különben 403-as hibát ad vissza.

FixedDeductionController
A fix kiadások kezelését végzi. A `store()` metódus új fix kiadást hoz létre, az `update()` módosítja, a `destroy()` törli. Tulajdonosi ellenőrzés minden műveletnél.

NotificationController
Az értesítések és emlékeztetők kezelését végzi. Hasonló CRUD működés, mint a többi kontroller.

AdminController
Az adminisztrátori funkciókat kezeli. A `users()` metódus listázza az összes felhasználót expenses_count és expenses_sum aggregációval. A `createUser()` új felhasználót hoz létre,

az `updateUser()` módosít, a `deleteUser()` törli (saját magát nem engedi törölni). A `stats()` összesített statisztikákat ad vissza.

## 2.7. Autentikáció – Laravel Sanctum

Az API hitelesítés Laravel Sanctum token-alapú módszerrel működik. A bejelentkezéskor a szerver egy plaintext tokent ad vissza, amelyet a kliens minden kérésnél az `Authorization: Bearer <token>` fejlécben küld el. A token a `personal_access_tokens` táblában kerül tárolásra, és kijelentkezéskor törlődik.

**Admin ellenőrzés:** Az `AdminMiddleware` ellenőrzi, hogy a bejelentkezett felhasználó `is_admin` mezője `true`-e. Ha nem, 403-as hibát ad vissza.

## 2.8. Továbbfejlesztési lehetőségek

- WebSocket / Server-Sent Events alapú valós idejű értesítések.
- Részletes audit log: ki, mikor, mit változtatott a rendszerben.
- PDF alapú havi költségvetési riport automatikus generálása.
- Kétfaktoros hitelesítés (2FA) bevezetése admin felhasználók számára.
- Automatikus adatbázis backup és restore funkció.
- Redis alapú cache-elés a gyakran lekérdezett adatokhoz.

---

# 3. Fejlesztői dokumentáció – Frontend

## 3.1. Mappa struktúra

```
src/
├── app/
│   ├── app.component.ts          # Root komponens (router-outlet + toast)
│   ├── app.config.ts             # Alkalmazás konfiguráció (route, HTTP interceptors)
│   ├── app.routes.ts             # Útvonalak definíciója
│   ├── calendar-calculator/      # Fő alkalmazás (naptár + költéskezelés)
│   │   ├── calendar-calculator.ts
│   │   ├── calendar-calculator.html
│   │   └── calendar-calculator.css
│   ├── components/
│   │   └── toast/                # Toast értesítések
│   │       ├── toast.ts
│   │       ├── toast.html
│   │       └── toast.css
│   ├── dashboard/                # Admin felület
│   │   ├── dashboard.ts
│   │   ├── dashboard.html
│   │   ├── dashboard.css
```

```
│   │   └── dashboard.spec.ts
│   ├── interceptors/
│   │   ├── auth.interceptor.ts       # Bearer token automatikus küldése
│   │   ├── error.interceptor.ts      # Hibakezelés és toast megjelenítés
│   │   ├── loading.interceptor.ts    # Loading állapot kezelése
│   │   └── loading.interceptor.spec.ts
│   ├── login/                        # Bejelentkezés / regisztráció / beállítások
│   │   ├── login.ts
│   │   ├── login.html
│   │   └── login.css
│   ├── models/
│   │   ├── api.model.ts              # API válasz és kérés típusok
│   │   ├── theme.model.ts            # Téma interfész definíció
│   │   └── user.model.ts             # Felhasználói adat típusok
│   ├── services/
│   │   ├── auth.guard.ts             # Route védelem (bejelentkezés, admin)
│   │   ├── budget.service.ts         # Fő szolgáltatás – API hívások, üzleti logika
│   │   ├── error.service.ts          # Toast kezelés
│   │   ├── error.service.spec.ts
│   │   ├── loading.service.ts        # Loading spinner állapot
│   │   └── theme.service.ts          # Téma váltás és CSS változók kezelése
│   └── welcome/                      # Kezdőlap
│       ├── welcome.ts
│       ├── welcome.html
│       └── welcome.css
├── environments/
│   ├── environment.ts                # Dev környezet (API URL, app név, verzió)
│   └── environment.prod.ts           # Produkciós környezet
├── index.html
├── main.ts                           # Bootstrap (bootstrapApplication)
└── styles.css                        # Globális stílusok, CSS változók
```

## 3.2. Fejlesztéshez használt eszközök és technológiák

| Eszköz / Technológia | Verzió | Felhasználás |
|---|---|---|
| Angular | 21+ | Frontend keretrendszer, standalone komponensek |
| TypeScript | 5.x | Típusos JavaScript |
| RxJS | 7.x | Reaktív programozás, HTTP hívások kezelése |
| CSS3 | – | Stílusozás, CSS változók, animációk |
| Angular CLI | 21+ | Projekt generálás, build, serve |
| Node.js | 18+ | Futtatási környezet |
| npm | 9+ | Csomagkezelő |
| Visual Studio Code | – | Fejlesztői szövegszerkesztő |

## 3.3. Komponensek

WelcomeComponent (`welcome/welcome.ts`)
A landing page komponens, amely az alkalmazás első betöltődéskor jelenik meg. Tartalmazza az alkalmazás rövid bemutatását, a bejelentkezés/regisztráció gombot, valamint a téma választó lehetőséget.

**Kulcsfunkciók:**
- Téma váltás (ThemeService integráció)
- Navigáció a bejelentkezési oldalra

LoginComponent (`login/login.ts`)
A bejelentkezési és regisztrációs űrlapot, valamint a kezdeti beállításokat kezeli. Két módot támogat: bejelentkezés és regisztráció. Sikeres autentikáció után megjeleníti a felhasználói beállításokat (fizetés, fix kiadások, értesítések).

**Kulcsfunkciók:**
- Bejelentkezés / regisztráció (`BudgetService.login`, `BudgetService.register`)
- Fizetés megadása és mentése
- Fix kiadások hozzáadása, szerkesztése, törlése
- Értesítések (emlékeztetők) hozzáadása és törlése
- Havi költségvetés kiszámítása (`salary - fix kiadások`)
- Téma váltás

TrackyComponent (`calendar-calculator/calendar-calculator.ts`)
A fő alkalmazási komponens, amely a naptár nézetet és a költéskezelést biztosítja. Itt történik a napi költések rögzítése, a naptár megjelenítése, valamint a statisztikák és diagramok kezelése.

**Kulcsfunkciók:**
- Naptár generálása (hétfővel kezdődő hét)
- Napi költések rögzítése kategóriák szerint (étel, bolt, cigi, szórakozás, kávé, utazás, ruházat, egészség, számlák, egyéb)
- Napi/heti/havi költési statisztikák megjelenítése
- Kategóriánkénti költési megoszlás (kördiagram)
- Heti limit figyelmeztetés (70% és 100% túllépés)
- Napi értesítések megjelenítése (esedékes számlák, emlékeztetők)
- Szerver státusz ellenőrzés (`/api/ping`)
- Hónap váltása (előző/következő)
- Nap szerkesztése (részletes nézet egy adott nap költéseihez)

DashboardComponent (`dashboard/dashboard.ts`)
Az adminisztrátori felület, amely csak admin jogosultsággal érhető el. A rendszer összes felhasználójának adatait, statisztikáit és kezelési lehetőségeit biztosítja.

**Kulcsfunkciók:**
- Összes felhasználó listázása (`/api/admin/users`)
- Új felhasználó létrehozása
- Felhasználó adatainak módosítása (fizetés, e-mail)

- Felhasználó törlése
- Rendszerstatisztikák megjelenítése (`/api/admin/stats`)
- Szerver státusz ellenőrzése
- Téma váltás

ToastComponent (`components/toast/toast.ts`)
Az értesítések megjelenítését végzi. Az `ErrorService` által kezelt toast üzeneteket jeleníti meg a képernyő tetején. Támogatja a success, warning, error és info típusokat.

## 3.4. Szervizek

BudgetService (`services/budget.service.ts`)
A frontend legfontosabb szolgáltatása. Kezeli a felhasználói adatokat, az API kommunikációt, az autentikációt és az üzleti logikát.

**Főbb felelősségek:**
- **Autentikáció:** `login()`, `register()`, `logout()` – Bearer token és user ID tárolása `localStorage`-ban.
- **Adatkezelés:** `loadUserData()`, `saveUserData()`, `updateSalary()` – API-n keresztüli adat szinkronizáció.
- **CRUD műveletek:** `addExpense()`, `addFixedDeduction()`, `addNotification()`, `removeFixedDeduction()`, `removeNotification()`, `updateFixedDeduction()`, `updateExpense()`.
- **Üzleti logika:**
  - `getDailyBudget()` – Napi költségvetés számítása a megmaradt összeg és a hátralévő napok alapján.
  - `getSpentThisMonth()` – Aktuális havi költés összesítése.
  - `getSpentThisWeek()` – Aktuális heti költés összesítése.
  - `getWeeklyLimitStatus()` – Heti limit állapotának ellenőrzése (70% figyelmeztetés, 100% túllépés).
  - `getTodaysNotifications()` – Mai napi esedékes értesítések lekérdezése.
  - `checkAndResetMonthlyExpenses()` – Havi költések automatikus nullázása új hónap kezdetekor.
- **Signal alapú reaktivitás:** A `userData` signal (`WritableSignal<UserData>`) biztosítja, hogy a komponensek automatikusan frissüljenek az adatváltozásokkor.

ThemeService (`services/theme.service.ts`)
A témaváltásért és a CSS változók dinamikus alkalmazásáért felelős. 8 előre definiált témát tartalmaz (Sötét, Világos, Naplemente, Őserdő, Tenger, Rózsa, Arany, Minimal).

**Kulcsfunkciók:**
- `setTheme(index)` – Téma váltása és mentése `localStorage`-ba.
- `applyTheme(theme)` – CSS változók beállítása a `:root` elemre (`--primary-color`, `--background`, stb.).
- `currentTheme` – Signal, amely a jelenleg aktív témát tárolja.

ErrorService (`services/error.service.ts`)

A hibák és sikeres műveletek toast értesítésként való megjelenítését kezeli.

**Kulcsfunkciók:**
- `show(message, type, duration)` – Toast üzenet megjelenítése.
- `handleError(error, fallbackMessage)` – HTTP hibák feldolgozása és megjelenítése.
- `handleSuccess(message)` – Sikeres művelet visszajelzése.
- `toasts` – Signal, amely a jelenleg megjelenített toast üzeneteket tartalmazza.

LoadingService (`services/loading.service.ts`)
A HTTP kérések alatt megjelenő loading spinner állapotát kezeli.

**Kulcsfunkciók:**
- `start()` / `stop()` – Kérésszámláló alapú loading állapot kezelése.
- `isLoading` – Signal, amely jelzi, hogy van-e folyamatban lévő kérés.

## 3.5. Útvonalak és route-védők

Az útvonalak az `app.routes.ts` fájlban vannak definiálva:

```typescript
export const routes: Routes = [
    { path: '', component: WelcomeComponent },
    { path: 'login', component: LoginComponent },
    { path: 'main', component: TrackyComponent, canActivate: [autoLoginGuard] },
    { path: 'admin', component: DashboardComponent, canActivate: [adminGuard] },
    { path: '**', component: WelcomeComponent }
];
```

| Útvonal | Komponens | Védelem | Leírás |
|---|---|---|---|
| `/` | WelcomeComponent | – | Kezdőlap |
| `/login` | LoginComponent | – | Bejelentkezés / regisztráció |
| `/main` | TrackyComponent | `autoLoginGuard` | Fő alkalmazás (csak bejelentkezve) |
| `/admin` | DashboardComponent | `adminGuard` | Admin felület (csak admin jogosultsággal) |
| `**` | WelcomeComponent | – | 404 – vissza a kezdőlapra |

Route védők (`services/auth.guard.ts`)

**autoLoginGuard:** Ellenőrzi, hogy a felhasználó be van-e jelentkezve (`BudgetService.isLoggedIn()`). Ha nem, átirányít a `/login` oldalra.

**adminGuard:** Ellenőrzi, hogy a felhasználó be van-e jelentkezve ÉS admin jogosultsággal rendelkezik-e (`BudgetService.isAdminUser()`). Ha nem, átirányít a `/main` oldalra.

## 3.6. Adatmodellek (TypeScript interfészek)

`user.model.ts`

```typescript
export interface FixedDeduction {
  id?: number;
  name: string;
  amount: number;
}

export interface Notification {
  id?: number;
  name: string;
  amount: number;
  day: number;
  recurring: boolean;
}

export interface Expense {
  id?: number;
  date: string;
  amount: number;
  description: string;
}

export interface UserData {
  email?: string;
  salary: number;
  fixedDeductions: FixedDeduction[];
  notifications: Notification[];
  expenses: Expense[];
}

export interface LoginResponse {
  success: boolean;
  userId?: string;
  token?: string;
  is_admin?: boolean;
  message?: string;
}
```

`theme.model.ts`

```typescript
export interface Theme {
```

```
  name: string;
  icon: string;
  primaryColor: string;
  secondaryColor: string;
  background: string;
  backgroundGradient: string;
  cardBg: string;
  textColor: string;
  textSecondary: string;
  borderColor: string;
  success: string;
  warning: string;
  danger: string;
  inputBg: string;
  isDark: boolean;
}
```

`api.model.ts`

```typescript
export interface ApiSuccessResponse {
  success: true;
  message?: string;
}

export interface PingResponse {
  status: string;
  timestamp: string;
}

export interface AdminUsersResponse {
  users: Record<string, Omit<any, 'password'>>;
  totalUsers: number;
  dataFileSize: number;
}
```

## 3.7. HTTP Interceptorok

Az interceptorok az `app.config.ts`-ben vannak regisztrálva a `provideHttpClient(withInterceptors([...]))` segítségével. A végrehajtási sorrend: `authInterceptor` → `loadingInterceptor` → `errorInterceptor`.

#### auth.interceptor.ts
Minden HTTP kéréshez automatikusan hozzáadja az `Authorization: Bearer <token>` fejlécet, ha a felhasználó be van jelentkezve (a token a `localStorage`-ból olvasódik).

loading.interceptor.ts
Minden HTTP kérés indításakor meghívja a `LoadingService.start()` metódust, és a kérés befejezésekor (finalize) a `LoadingService.stop()` metódust. Ez biztosítja, hogy a loading spinner csak akkor tűnjön el, amikor az összes párhuzamos kérés befejeződött.

error.interceptor.ts
HTTP hibák esetén a megfelelő hibaüzenetet állítja össze a státuszkód alapján:
- **0** – A szerver nem elérhető
- **400** – Hibás kérés
- **401** – Nincs jogosultság
- **404** – A kért erőforrás nem található
- **500** – Szerver hiba

A hibaüzenetet az `ErrorService.show()` metódussal jeleníti meg toast formában.

## 3.8. Továbbfejlesztési lehetőségek

- **PWA (Progressive Web App)** támogatás: Offline működés Service Workerrel.
- **Animációk és átmenetek:** Angular Animations használata oldalváltásokhoz és elemek megjelenéséhez.
- **Reszponzív mobil optimalizálás:** Érintésre optimalizált felület, swipe gesztusok.
- **Többnyelvűség (i18n):** Angular i18n vagy ngx-translate integrálása.
- **Költségvetési célok beállítása:** Havi/éves megtakarítási célok és azok nyomon követése.
- **Export funkció:** CSV/Excel export a költési adatokból.
- **Grafikonok és diagramok:** Chart.js integrálása részletesebb statisztikákhoz.

---

# 4. Felhasználói kézikönyv

## 4.1. Regisztráció és bejelentkezés

1. Nyisd meg a böngészőben az alkalmazást: `http://localhost:4200`
2. A kezdőlapon kattints a **Bejelentkezés** gombra.
3. Ha még nincs fiókod, kattints az **Új fiók létrehozása** linkre.
4. Töltsd ki a kötelező mezőket:
   - **Felhasználónév** (legalább 1 karakter)
   - **Jelszó** (legalább 4 karakter)
   - **E-mail** (regisztrációnál)
   - **Jelszó megerősítése** (regisztrációnál)
5. Kattints a **Bejelentkezés** vagy **Regisztráció** gombra.
6. Sikeres autentikáció után automatikusan megjelennek a **Beállítások**.

### 4.2. Beállítások (fizetés, fix kiadások, értesítések)

A bejelentkezés után a következő beállításokat kell megadni:

Fizetés megadása
- Add meg a havi nettó fizetésedet forintban.
- Ez az összeg lesz az alapja a napi költségvetés számításának.

Fix kiadások hozzáadása
- Kattints a **+ Fix kiadás hozzáadása** gombra.
- Add meg a kiadás nevét (pl. "Albérlet", "Rezsi") és az összegét.
- A fix kiadások automatikusan levonásra kerülnek a havi költségvetésből.
- Gyors hozzáadás: kattints az előre definiált gombokra (Albérlet, Rezsi, Telefon, Internet).

Értesítések (emlékeztetők) beállítása
- Kattints a **+ Értesítés hozzáadása** gombra.
- Add meg az értesítés nevét, összegét és a hónap napját (1-31).
- Az értesítések ismétlődőek (minden hónapban ugyanazon a napon).
- A fő képernyőn a mai napi értesítések kiemelten megjelennek.

Mentés és folytatás
- Kattints a **Mentés és folytatás** gombra.
- Az adatok automatikusan szinkronizálódnak a szerverrel.

## 4.3. Fő képernyő – Naptár és költéskezelés

A fő képernyő (`/main`) a következő elemeket tartalmazza:

Naptár
- A hónap napjai hétfővel kezdődően jelennek meg.
- A mai nap kiemelten szerepel.
- Minden nap alatt látható az aznapi összesített költés.
- Kattints egy napra a részletes nézet megnyitásához.

Költés rögzítése
1. Válassz dátumot (alapértelmezett a mai nap).
2. Válassz kategóriát (Étel, Bolt, Cigi, Szórakozás, Kávé, Utazás, Ruházat, Egészség, Számlák, Egyéb).
3. Add meg az összeget forintban.
4. Opcionálisan adj meg leírást.
5. Kattints a **Költés rögzítése** gombra.

Statisztikák
- **Napi költségvetés:** A megmaradt összeg osztva a hátralévő napok számával.
- **Havi költés:** Az aktuális hónapban rögzített költések összege.
- **Heti limit állapot:** Figyelmeztetés, ha a heti költés eléri a 70%-ot vagy meghaladja a 100%-ot.
- **Kategóriánkénti megoszlás:** Kördiagram a költések kategóriánkénti eloszlásáról.
- **Top költések:** Az 5 legnagyobb költés listája.

Napi részletes nézet
- Kattints egy napra a naptárban.
- Megjelennek az adott napi költések részletesen.
- Lehetőség van költés szerkesztésére és törlésére.

Értesítések
- A képernyő tetején megjelennek a mai napi esedékes értesítések (pl. "Ma esedékes: Rezsi – 25 000 Ft").

Egyéb funkciók
- **Hónap váltása:** Előző és következő hónap gombokkal.
- **Beállítások:** Visszalépés a fizetés és fix kiadások módosításához.
- **Kijelentkezés:** Kilépés és adatok törlése a böngészőből.

## 4.4. Adminisztrátori felület

Az admin felület (`/admin`) csak admin jogosultsággal érhető el.

Elérés
1. Jelentkezz be az admin felhasználóval (alapértelmezett: `admin` / `admin123`).
2. A fő képernyőn kattints az **Admin** gombra, vagy navigálj közvetlenül a `/admin` útvonalra.

Funkciók
- **Felhasználók listája:** Az összes regisztrált felhasználó megjelenítése (név, e-mail, fizetés, költések száma, összes költés).
- **Új felhasználó létrehozása:** Felhasználónév, jelszó, e-mail és fizetés megadásával.
- **Felhasználó módosítása:** Fizetés és e-mail cím szerkesztése.
- **Felhasználó törlése:** Felhasználó végleges eltávolítása a rendszerből (saját magát nem lehet törölni).
- **Statisztikák:** Összesített adatok (felhasználók száma, átlagfizetés, szerver státusz).

## 4.5. Téma váltás

Az alkalmazás 8 különböző szín témát támogat:

| Téma | Ikon | Jellemzők |
|---|---|---|
| Sötét | 🌙 | Alapértelmezett, kékes-lila árnyalatok |
| Világos | ☀️ | Világos háttér, sötét szöveg |
| Naplemente | 🌅 | Meleg, narancs-piros tónusok |
| Őserdő | 🌲 | Zöld, természetes színek |
| Tenger | 🌊 | Kék, óceán ihlette |
| Rózsa | 🌸 | Rózsaszín, romantikus |
| Arany | ✨ | Arany, luxus hatás |
| Minimal | 💎 | Egyszerű, fehér alap |

**Témaváltás:** Kattints a képernyő jobb felső sarkában lévő téma ikonra, és válaszd ki a kívánt témát. A választás automatikusan mentődik a böngészőben.

---

# 5. Telepítési útmutató

## 5.1. Követelmények

| Szoftver | Min. verzió | Letöltés |
|---|---|---|
| PHP | 8.2 | https://windows.php.net/download |
| Composer | 2.x | https://getcomposer.org/download/ |
| MySQL / MariaDB | 8.0+ | XAMPP, WAMP vagy önálló |
| Node.js | 18+ | https://nodejs.org/ |
| npm | 9+ | Csomagkezelő (Node.js-szel együtt jön) |
| Angular CLI | 21+ | `npm install -g @angular/cli` |

**Szükséges PHP kiterjesztések (php.ini):**
```ini
extension=pdo_mysql
extension=zip
extension=fileinfo
extension=openssl
```

**Backend függőségek (automatikusan települnek):**
- Laravel 12.x
- Laravel Sanctum 4.x
- Vite 7.x
- Tailwind CSS 4.x

## 5.2. MySQL adatbázis létrehozása

Hozd létre a `trackmate` adatbázist MySQL-ben:

```sql
CREATE DATABASE trackmate CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

## 5.3. Laravel backend telepítése

1. **Környezeti változók beállítása:**
   Nyisd meg a `trackmate-fullstack/backend/.env` fájlt, és állítsd be:
   ```env
   ```

```
APP_NAME=Tracky
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=trackmate
DB_USERNAME=root
DB_PASSWORD=
```

2. **Composer függőségek telepítése:**
   ```powershell
   cd C:\ge\trackmate\trackmate-fullstack\backend
   composer install
   ```

3. **Alkalmazás kulcs generálása:**
   ```powershell
   php artisan key:generate
   ```

4. **Migráció és seedelés:**
   ```powershell
   php artisan migrate --seed
   ```
   Ez létrehozza az összes táblát és feltölti a super admin felhasználót.

5. **Laravel szerver indítása:**
   ```powershell
   php artisan serve
   ```
   A szerver elérhető: **http://localhost:8000**

## 5.4. Angular frontend telepítése

1. **Függőségek telepítése:**
   Nyiss egy új terminált:
   ```powershell
   cd C:\ge\trackmate
   npm install
   ```

2. **Fejlesztői szerver indítása:**
   ```powershell
   ng serve
   ```

A frontend elérhető: **http://localhost:4200**

## 5.5. Alapértelmezett belépési adatok (seedelés után)

| Szerep | Felhasználónév | Jelszó |
|---|---|---|
| Super Admin | `admin` | `admin123` |
| Teszt user | `test` | `test1234` |

## 5.6. Fejlesztői parancsok

#### Laravel
```powershell
# Szerver indítása
php artisan serve

# Vite dev szerver (CSS/JS fejlesztéshez)
npm run dev

# Új migráció
php artisan make:migration create_tabla_neve

# Migráció futtatása
php artisan migrate

# Adatbázis újraseedelése
php artisan migrate:fresh --seed

# Route-ok listázása
php artisan route:list

# Tinker shell (adatbázis debug)
php artisan tinker
```

#### Angular
```powershell
# Szerver indítása
ng serve

# Szerver indítása megnyitott böngészővel
ng serve --open

# Build (fejlesztői)
ng build

# Build (produkciós)
```

```
ng build --configuration production

# Új komponens
ng generate component komponens-neve

# Új szolgáltatás
ng generate service services/szolgaltatas-neve

# Unit tesztek futtatása
ng test

# Linting (kódstílus ellenőrzés)
ng lint

# E2E tesztek futtatása
ng e2e

# Adott projekt buildere
ng build --project admin-app
```

**Megjegyzés:** A backend Tailwind CSS 4.x-et használ fejlesztéshez. A CSS automatikusan recompilálódik a `npm run dev` parancs futása közben.

### 5.7. Hibaelhárítás

#### CORS hiba ("Access to XMLHttpRequest has been blocked")
**Probléma:** A frontend nem tud kommunikálni a backenddel.

**Megoldás:**
1. Ellenőrizd a Laravel `.env` fájlban:
```env
APP_URL=http://localhost:8000
SANCTUM_STATEFUL_DOMAINS=localhost:4200,127.0.0.1:4200
```

2. Ellenőrizd az `app/Http/Kernel.php` fájlban a CORS beállítást:
```php
'cors' => \App\Http\Middleware\Cors::class,
```

3. Indítsd újra a Laravel szervert:
```powershell
php artisan serve
```

#### "No such file or directory" vendor mappában
**Probléma:** A Composer függőségek hiányoznak.

**Megoldás:**
```powershell
cd C:\ge\trackmate\trackmate-fullstack\backend
composer install
```

#### Token érvénytelenség ("401 Unauthorized")
**Probléma:** A bejelentkezés után nem működnek az API kérések.

**Megoldás:**
1. Töröld a böngésző `localStorage`-át:
   - Nyomj `F12` → `Application` tab → `Local Storage` → Kattints `http://localhost:4200` → `Clear`
2. Jelentkezz be újra
3. Ellenőrizd a böngésző konzolban (F12 → Console) az API válaszokat

MySQL kapcsolat sikertelen ("SQLSTATE[HY000] [2002]")
**Probléma:** A Laravel nem tud csatlakozni az adatbázishoz.

**Megoldás:**
1. Indítsd el a MySQL szervert (XAMPP Control Panel → MySQL Start)
2. Ellenőrizd, hogy a `trackmate` adatbázis létezik:
```powershell
mysql -u root -p
> SHOW DATABASES;
> CREATE DATABASE trackmate CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

3. Ellenőrizd a `.env` fájl adatbázis beállításait:
```env
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=trackmate
DB_USERNAME=root
DB_PASSWORD=
```

4. Futtasd a migrációkat:
```powershell
php artisan migrate --seed
```

#### Port már használatban van (3000, 4200, 8000)
**Probléma:** A szerver nem indul el, mert az port már foglalt.

**Megoldás - Windows PowerShell:**

```powershell
# Laravel (8000-es port)
php artisan serve --port=8001

# Angular (4200-es port)
ng serve --port=4201

# Folyamat keresése port alapján
netstat -ano | findstr :8000
taskkill /PID <PID> /F
```

#### "npm: command not found"
**Probléma:** Node.js nincs telepítve vagy nem érhető el az PATH-ban.

**Megoldás:**
1. Töltsd le a Node.js-t: https://nodejs.org/
2. Telepítsd és indítsd újra a terminált
3. Ellenőrizd a telepítést:
```powershell
node --version
npm --version
```

#### "ng: command not found"
**Probléma:** Az Angular CLI nincs telepítve globálisan.

**Megoldás:**
```powershell
npm install -g @angular/cli
# Ellenőrzés
ng version
```

#### NPM csomagok telepítésének problémái
**Probléma:** `npm install` során hibák fordulnak elő.

**Megoldás:**
```powershell
# Cache törlése
npm cache clean --force

# node_modules és package-lock.json törlése
Remove-Item -Recurse -Force node_modules
Remove-Item package-lock.json

# Újra telepítés
npm install
```

```

#### PHP: "Class not found" hiba
**Probléma:** Egy Laravel class vagy interface nem található.

**Megoldás:**
```powershell
cd C:\ge\trackmate\trackmate-fullstack\backend
composer dump-autoload
php artisan clear-cache
php artisan config:clear
```

#### Angular build hiba ("Cannot find module")
**Probléma:** TypeScript fordítási hiba.

**Megoldás:**
```powershell
# node_modules ujratelepítése
npm install

# Gyorsítótár törlése
rm -recurse -force .angular

# Újra buildelés
ng build
```

#### Bejelentkezés után a fő oldal üres
**Probléma:** A TrackyComponent nem jelenik meg helyesen.

**Megoldás:**
1. Nyomj `F12` → `Console` és keress error üzenetet
2. Ellenőrizd a szerver `localhost:8000/api/me` végpont válaszát Postmanben
3. Szerver logja: `php artisan serve` terminálban
4. Töröld újra a `localStorage`-at és jelentkezz be

 Admin felület nem érhető el
**Probléma:** "403 Forbidden" vagy átirányítás a fő oldalra.

**Megoldás:**
1. Ellenőrizd, hogy admin felhasználóval vagy-e bejelentkezve
2. Adatbázist ellenőrizd: `SELECT * FROM users WHERE is_admin = 1;`
3. Ha szükséges, frissítsd az adatbázist:
```sql
UPDATE users SET is_admin = 1 WHERE name = 'admin';
```
```

Szerver nem indul el, "Address already in use"
**Probléma:** Port már foglalt.

**Megoldás - Laravel:**
```powershell
# Más port használata
php artisan serve --host=localhost --port=8001

# Ha nem segít, proc keresése
netstat -ano | findstr :8000
taskkill /PID <folyamat_id> /F
```

Az env fájl nem betöltődik
**Probléma:** Módosítások után a Laravel továbbra is régi értékeket használ.

**Megoldás:**
```powershell
php artisan config:clear
php artisan cache:clear
php artisan config:cache
```

TypeScript típus hibák
**Probléma:** Az IDE mutat hibákat, de az alkalmazás működik.

**Megoldás:**
1. Frissítsd a TypeScript verziót:
```powershell
npm install --save-dev typescript@latest
```

2. Ellenőrizd a `tsconfig.json` fájlt:
```json
{
  "compilerOptions": {
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  }
}
```

## 5.8. Gyorsított telepítési útmutató (lépésről-lépésre)

Ez a leggyorsabb útmutató a Tracky teljes futtatásához.

1. lépés: Szükséges szoftverek telepítése
Ellenőrizd, hogy telepítve vannak-e:
```powershell
php --version       # PHP 8.2+
node --version      # Node.js 18+
npm --version       # npm 9+
composer --version  # Composer 2.x
mysql --version     # MySQL 8.0+
```

Ha valami hiányzik, letölthető:
- **PHP:** https://windows.php.net/download/
- **Node.js:** https://nodejs.org/
- **Composer:** https://getcomposer.org/download/
- **MySQL:** XAMPP vagy MariaDB

2. lépés: MySQL adatbázis
Nyiss egy PowerShell-t admin módban:
```powershell
mysql -u root -p
# Jelszót kérdez (alapértelmezett: üres)

> CREATE DATABASE trackmate CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
> EXIT;
```

3. lépés: Backend telepítése
```powershell
cd C:\ge\trackmate\trackmate-fullstack\backend

# .env másolása
Copy-Item .env.example .env

# Módosítsd az .env fájlt az alábbi értékekkel:
# DB_DATABASE=trackmate
# DB_USERNAME=root
# DB_PASSWORD=

# Függőségek
composer install

# Alkalmazás kulcs
php artisan key:generate

# Adatbázis előkészítés
php artisan migrate --seed
```

```
# Szerver indítása
php artisan serve
```

**Backend ellenőrzése:** Nyisd meg böngészőben: `http://localhost:8000/api/ping`

4. lépés: Frontend telepítése

Nyiss egy **új** PowerShell ablakot:

```powershell
cd C:\ge\trackmate

# Függőségek
npm install

# Szerver indítása
ng serve --open
```

A böngésző automatikusan megnyílik a `http://localhost:4200`-on.

5. lépés: Bejelentkezés

Használd az alapértelmezett adatokat:

- **Felhasználónév:** `admin`
- **Jelszó:** `admin123`

6. lépés: Beállítások kitöltése

1. Add meg a fizetésedet
2. Add hozzá a fix kiadásokat (pl. Albérlet, Rezsi)
3. Állítsd be az értesítéseket
4. Kattints a **Mentés és folytatás** gombra

**Kész!** Az alkalmazás teljes mértékben működik.

---

Gyors debug parancsok

Ha valami nem működik:

```powershell
# Backend hibák
cd C:\ge\trackmate\trackmate-fullstack\backend
php artisan tinker
> User::all();  # Felhasználók listázása
> exit;

# Frontend hibák
```

```
cd C:\ge\trackmate
ng lint
ng build

# Port elfoglaltsága
netstat -ano | findstr :8000   # Laravel port
netstat -ano | findstr :4200   # Angular port

# Cache törlés
cd C:\ge\trackmate\trackmate-fullstack\backend
php artisan cache:clear
php artisan config:clear
```

---

## 6. Összefoglalás

A Tracky egy modern, teljes körű költségvetés-kezelő alkalmazás, amely Laravel 12 backenddel és Angular 21+ frontenddel készült. A rendszer a következő főbb jellemzőkkel rendelkezik:

- **Biztonságos autentikáció:** Laravel Sanctum token-alapú hitelesítés.
- **Teljes CRUD műveletek:** Felhasználók, költések, fix kiadások és értesítések kezelése.
- **Adminisztrátori felület:** Felhasználók és rendszer statisztikák kezelése.
- **Naptár nézet:** Vizualizált havi költések napi bontásban.
- **Automatikus számítások:** Napi költségvetés, heti limit figyelmeztetések.
- **Többszínű témák:** 8 különböző megjelenés.
- **Reszponzív dizájn:** Modern, felhasználóbarát felület.

A projekt jól strukturált, modularizált kód bázissal rendelkezik, amely könnyen továbbfejleszthető és karbantartható. A dokumentáció részletesen bemutatja az architektúrát, az API végpontokat, a frontend komponenseket és a telepítési lépéseket, így a rendszer reprodukálható és továbbfejleszthető.

**Jövőbeli fejlesztési irányok:** PWA támogatás, valós idejű értesítések, PDF riport generálás, többnyelvűség, és részletes analitikai dashboard.

---

## 7. Telepítési Checklist

Használd ezt a check listet a sikeres telepítéshez:

Szoftverek és előfeltételek
- [ ] PHP 8.2 vagy újabb telepítve
- [ ] MySQL 8.0 vagy újabb futó

- [ ] Node.js 18+ telepítve
- [ ] npm 9+ telepítve
- [ ] Composer 2.x telepítve
- [ ] Visual Studio Code (vagy másik IDE)
- [ ] Git telepítve

### MySQL adatbázis
- [ ] MySQL szerver elindítva
- [ ] `trackmate` adatbázis létrehozva
- [ ] UTF-8 karakter kódolás beállítva

### Laravel Backend
- [ ] `trackmate-fullstack/backend` mappában
- [ ] `.env` fájl létrehozva és módosítva
  - [ ] `DB_DATABASE=trackmate`
  - [ ] `DB_USERNAME` és `DB_PASSWORD` beállítva
  - [ ] `APP_URL=http://localhost:8000`
- [ ] `composer install` futott
- [ ] `php artisan key:generate` futott
- [ ] `php artisan migrate --seed` futott
- [ ] `php artisan serve` elindult
- [ ] `http://localhost:8000/api/ping` működik

### Angular Frontend
- [ ] `C:\ge\trackmate` mappában
- [ ] `npm install` futott
- [ ] `ng serve` elindult
- [ ] `http://localhost:4200` betöltődik

### Tesztelés
- [ ] Admin bejelentkezés működik (`admin` / `admin123`)
- [ ] Fizetés beállítható
- [ ] Költés rögzíthető
- [ ] Naptár jelenik meg
- [ ] Téma váltható
- [ ] Admin panel elérhető (`/admin`)
- [ ] Kijelentkezés működik

### Fejlesztési készenlét
- [ ] Gitignore megfelelően beállított
- [ ] Egy Git branch a fejlesztéshez
- [ ] VS Code extensions telepítve:
  - [ ] PHP Intelephense
  - [ ] Angular Language Service
  - [ ] ES7+ React/Redux/React-Native snippets
  - [ ] Thunder Client (API teszteléshez)

### Termelési Deploy (opcionális)

- [ ] `.env.production` fájl elkészítve
- [ ] Laravel build parancs: `php artisan build`
- [ ] Angular build parancs: `ng build --configuration production`
- [ ] Szerver beállítások (https, domain, DNS)
- [ ] Adatbázis backup stratégia